

# Add a pmr alias for std::stacktrace

Document #: P2301R1  
Date: 2021-06-14  
Project: Programming Language C++  
Audience: LEWG, LWG  
Reply-to: Steve Downey  
<[sdowney2@bloomberg.net](mailto:sdowney2@bloomberg.net), [sdowney@gmail.com](mailto:sdowney@gmail.com)>

Abstract: This paper proposes to add an alias in the pmr namespace defaulting the allocator used by the std::basic\_stacktrace template to pmr::allocator. No changes to the api of std::stacktrace are necessary.

## § 1 Changes

### § 1.1 Changes since R0

Removed unneeded “std:” in the pmr alias.

```
namespace pmr {  
    using stacktrace = std::basic_stacktrace<polymorphic_allocator<stacktrace_entry>>;  
}
```

Expanded Motivation.

## § 2 Before / After Table

| Before                                                                                                                                                                                                                                                                | After                                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>char buffer[1024];<br/><br/>std::pmr::monotonic_buffer_resource pool{<br/>    std::data(buffer), std::size(buffer)};<br/><br/>std::basic_stacktrace&lt;<br/>    std::pmr::polymorphic_allocator&lt;std::stacktrace_entry&gt;&gt;<br/>    trace{&amp;pool};</pre> | <pre>char buffer[1024];<br/><br/>std::pmr::monotonic_buffer_resource<br/>    pool{<br/>        std::data(buffer),<br/>        std::size(buffer)};<br/><br/>std::pmr::stacktrace trace{&amp;pool};</pre> |

## § 3 Motivation

The type std::basic\_stacktrace is a class template with an allocator type parameter, the allocator is fixed to be std::allocator in the std::stacktrace alias, and it models *AllocatorAwareContainer*. All of the work to enable a pmr using type has been done, except for actually specifying the existence of the template alias std::pmr::stacktrace with a polymorphic\_allocator.

In general a template should have an alias in the pmr namespace when the primary template supports the allocator model and the allocator template parameter is defaulted to be std::allocator. Providing the pmr alias is a convenience for changing the default.

Not having the `pmr` alias makes `std::stacktrace` inconsistent with the other types that support using a `polymorphic_allocator`. Programmers that wish to use types supporting polymorphic memory resources should expect to find those types having a `pmr` alias that changes the default. Although the burden is often low for writing such an alias, it is not zero.

In addition, writing into a fixed contiguous buffer can improve performance allowing this facility to be used safely in more contexts.

## § 4 Proposed Wording

Modify **20.21.2** [\[stacktrace.syn\]](#) as indicated

```
namespace std {
    // [stacktrace.entry], class stacktrace_entry
    class stacktrace_entry;

    // [stacktrace.basic], class template basic_stacktrace
    template<class Allocator>
        class basic_stacktrace;

    // basic_stacktrace typedef names
    using stacktrace = basic_stacktrace<allocator<stacktrace_entry>>;

    // [stacktrace.basic.nonmem], non-member functions
    template<class Allocator>
        void swap(basic_stacktrace<Allocator>& a, basic_stacktrace<Allocator>& b)
            noexcept(noexcept(a.swap(b)));

    string to_string(const stacktrace_entry& f);

    template<class Allocator>
        string to_string(const basic_stacktrace<Allocator>& st);

    template<class charT, class traits>
        basic_ostream<charT, traits>&
            operator<<(basic_ostream<charT, traits>& os, const stacktrace_entry& f);

    template<class charT, class traits, class Allocator>
        basic_ostream<charT, traits>&
            operator<<(basic_ostream<charT, traits>& os, const basic_stacktrace<Allocator>& st);

    namespace pmr {
        using stacktrace = basic_stacktrace<polymorphic_allocator<stacktrace_entry>>;
    }

    // [stacktrace.basic.hash], hash support
    template<class T> struct hash;
    template<> struct hash<stacktrace_entry>;
    template<class Allocator> struct hash<basic_stacktrace<Allocator>>;
}
```

[Compiler Explorer Link](#)